UNIVERSAL CREATIVE ENGINE (UCE)

Software Requirements Specification

Version 0.1 - Architecture Baseline

Purpose

Define the product, architectural, security, privacy, testing, data, interface, and V0 implementation requirements for an AI-native Universal Creative Engine that unifies semantic video editing, VFX, 3D/4D scene understanding, audio, procedural operations, generative models, specialist agents, deterministic controls, and self-repair within a local-first architecture.

Item	Description
Document status	Working baseline for team review
Version	0.1
Date	2026-09-15
Classification	Confidential working draft
Primary audience	Product, engineering, ML research, security, QA, infrastructure, design
Normative language	MUST / SHOULD / MAY

Document Control

Control	Value
Document owner	UCE core team
Approval authority	Project technical lead / architecture review board
Change control	Material architectural changes require an ADR and version bump
Baseline	v0.1

Revision History

Version	Date	Author/Owner	Summary
0.1	2026-09-15	UCE core team	Initial SRS and architecture baseline.

Normative Terms

- MUST / SHALL: mandatory for conformance.
- SHOULD: strongly recommended; deviations require documented rationale.
- MAY: optional or implementation-dependent.
- Local-first: user media and derived data remain on-device unless policy explicitly permits external execution.
- Entity: a persistent semantic object, person, sound source, environment element, event participant, or other identifiable element within the UCE World Graph.

Contents

1. Executive Summary
2. Purpose and Scope
3. Product Vision and Principles
4. Stakeholders and User Classes
5. System Context
6. Architectural Overview
7. Functional Requirements
8. Non-Functional Requirements
9. Privacy Requirements
10. Security Requirements
11. Testing and Verification
12. Data and Storage Architecture
13. Universal Entity Identity
14. World Graph and Dependency Model
15. Media and GPU Pipeline
16. Model Runtime and Router
17. Plugin and Extension Architecture
18. Agent Architecture
19. Audio World
20. Quality and Self-Repair

21. Provenance and Auditability
22. Interfaces and Standards
23. V0 Scope and Acceptance Criteria
24. Roadmap
25. Risk Register
26. Traceability Matrix
27. References

1. Executive Summary

UCE is intended to become an AI-native creative operating system rather than a conventional video editor. Its core technical thesis is that recorded or generated media should be represented as a persistent, editable world containing entities, geometry, motion, events, audio, relationships, dependencies, uncertainty, and provenance. Users should be able to select semantic entities, express desired changes in natural language or through deterministic controls, and have UCE plan and execute operations while preserving all locked state.

The architecture therefore combines a semantic World Graph, Universal Entity Identity (UEI), OpenUSD-based scene state, OpenTimelineIO editorial state, deterministic and generative execution paths, content-addressable storage, GPU-native media surfaces, model routing, specialist agents, policy enforcement, quality critics, localized self-repair, provenance, and continuous security/privacy/testing gates.

The first production milestone is deliberately narrow: import a real video, create a project, select and persistently track a semantic entity, remove that entity using reconstruction-first processing, verify the result, repair localized defects, and export the result without violating local-first privacy or non-destructive project requirements.

2. Purpose and Scope

2.1 Purpose

This SRS provides the requirements baseline for product planning, architecture, engineering, ML research, security, QA, infrastructure, and design. It is intended to be sufficiently precise for team decomposition and implementation planning.

2.2 In Scope

- AI-native video and media transformation.
- Semantic entity selection and tracking.
- Object removal, replacement, insertion, and dependency-aware editing.
- Persistent World Graph and Universal Entity Identity.
- Non-destructive operation graph and semantic undo/branching.
- Hybrid deterministic, procedural, simulation, and generative execution.
- 3D/4D reconstruction interfaces and OpenUSD integration.
- Audio understanding and cross-modal dependency linking.
- Local-first execution with optional cloud compute.
- Security, privacy, provenance, auditability, plugin sandboxing, and continuous testing.
- Open plugin/model capability protocol and industry-standard interoperability.

2.3 Out of Scope for V0

- Training a frontier-scale proprietary text-to-video foundation model.
- Reimplementing mature codecs, color science, scene formats, or physics engines from scratch.
- Full Blender/Houdini/Nuke feature parity.
- Long-form autonomous filmmaking.
- General-purpose copyright-detection evasion or platform fingerprint bypass.
- Mass-market collaboration or enterprise administration before core local editing is stable.

3. Product Vision and Principles

ID	Principle	Requirement
P-01	World-first, not frame-first	UCE SHALL model media as persistent entities and relationships over time rather than as isolated frames.
P-02	Semantic selection	Users SHALL be able to select meaning (person, object, shadow, sound, event) rather than only pixels.
P-03	Procedural before destructive	Editable procedural representations SHOULD be preferred over baked irreversible results.
P-04	Deterministic where possible	UCE SHALL use geometry, physics, transforms, compositing, and signal processing where they can provide reliable results.
P-05	Generation where necessary	Generative models SHALL be treated as replaceable workers for missing or non-computable information.
P-06	Human override	AI-created operations SHALL expose editable controls, masks, paths, parameters, or nodes whenever technically feasible.
P-07	Local-first privacy	No user media SHALL leave the device unless an explicit project policy permits external execution.
P-08	Self-verifying	Generative and complex transformations SHALL be evaluated by quality checks before final acceptance.
P-09	Open ecosystem	UCE SHOULD interoperate with established standards and support an extensible capability/plugin system.
P-10	Evidence-driven development	Major capability claims SHALL be benchmarked and regression-tested.

4. Stakeholders and User Classes

Item	Description
Beginner creator	Uses natural language, semantic selection, guided presets, and AI-assisted workflows.
Professional editor/VFX artist	Requires deterministic controls, timeline precision, node graphs, masks, metadata, color, and plugin

Item	Description
	compatibility.
3D/animation artist	Requires scene, camera, geometry, rig, materials, lighting, simulation, and render interoperability.
Audio professional	Requires semantic audio sources, separation, synchronization, spatial positioning, and export control.
ML researcher	Requires model adapters, reproducible experiments, benchmarks, manifests, and swap-in components.
Plugin/model developer	Requires stable capability APIs, sandboxing, permissions, packaging, signing, and diagnostics.
Security/privacy administrator	Requires policy enforcement, offline guarantees, audit logs, key management, and deployment controls.
Enterprise/studio operator	Requires private deployment, distributed compute, access control, collaboration, and predictable reproducibility.

5. System Context

UCE shall be organized around an authoritative project state and replaceable execution backends.

User / UI / AI Intent
|
Security + Privacy Policy
|
Director / Operation Planner
|
Operation + Dependency Graph
|
Semantic World Graph / UEI
|
Deterministic | Perception | Simulation | Generative
|
GPU / CPU / Local / Cloud Execution
|
Quality Critics + Localized Repair
|
Renderer / Export / Provenance

6. Architectural Overview

6.1 Authoritative State

- OpenUSD SHOULD represent scene hierarchy, geometry, transforms, cameras, lights, animation, materials, and UCE-applied schemas.
- OpenTimelineIO SHOULD represent editorial timing and timeline structure.
- UCE semantic schemas SHALL represent entity IDs, semantic labels, confidence, dependencies, locks, events, and provenance references.
- A semantic index MAY store vector embeddings, search indexes, histories, and high-volume analysis metadata not suitable for direct scene storage.

- Immutable media and derived artifacts SHALL be stored through a content-addressable store (CAS).

6.2 Core Logical Layers

Item	Description
Experience	Timeline, semantic viewport, node/compositor views, World view, command interface.
Agent intelligence	Director, specialist planning agents, quality agents.
Policy	Privacy, security, permissions, trust level, execution policy.
Semantic world	UEI, relationships, events, edit locks, dependencies, uncertainty.
Execution	Operation DAG, dependency invalidation, caching, workflow durability.
Representations	USD/mesh, skeleton, point/depth, Gaussian/neural, OpenVDB, audio objects.
Capabilities	Perception, deterministic media operations, generation, simulation, compositing.
Compute	CPU, GPU, CUDA/Vulkan/Metal/DirectX, local/cloud/farm.
Storage	CAS, immutable assets, snapshots, proxies, checkpoints, provenance.

7. Functional Requirements

ID	Priority	Requirement
FR-001	MUST	Create, open, save, version, and recover non-destructive UCE projects.
FR-002	MUST	Import common professional and consumer video/audio/image formats through a mature media stack such as FFmpeg.
FR-003	MUST	Extract and preserve timing, frame rate, resolution, pixel format, color metadata, audio streams, and source provenance.
FR-004	MUST	Provide interactive semantic selection of people, objects, regions, and supported audio sources.
FR-005	MUST	Assign a persistent Universal Entity Identity to selected or discovered semantic entities.
FR-006	MUST	Track entities across frames and maintain confidence/history through occlusion where supported.
FR-007	MUST	Support non-destructive object

ID	Priority	Requirement
		removal using reconstruction-first processing and generation only for missing information.
FR-008	SHOULD	Support object replacement with preservation of unrelated scene state.
FR-009	SHOULD	Support object insertion with depth/occlusion aware placement.
FR-010	MUST	Represent each transformation as an explicit operation with inputs, outputs, parameters, dependencies, provenance, and undo state.
FR-011	MUST	Support semantic locks including identity, camera, geometry, motion, lighting, temporal, and audio locks.
FR-012	MUST	Invalidate and recompute dependent results when authoritative entity state changes.
FR-013	MUST	Run quality checks on supported complex/generative operations and record defect locations/confidence.
FR-014	MUST	Support localized repair without requiring full-shot regeneration when the defect can be isolated.
FR-015	SHOULD	Support natural-language commands that compile to approved operation-graph actions.
FR-016	MUST	Prevent agents from directly mutating project files outside the validated operation API.
FR-017	MUST	Support local-only processing mode with zero external media transfer.
FR-018	SHOULD	Support optional cloud/model-provider execution when explicitly authorized by policy.
FR-019	MUST	Maintain audit logs for privileged operations, model execution, external calls, and exports.
FR-020	SHOULD	Embed or export privacy-filtered provenance using C2PA-compatible workflows where enabled.
FR-021	SHOULD	Expose conventional deterministic editing controls in addition to AI

ID	Priority	Requirement
		commands.
FR-022	SHOULD	Support plugin/model capability discovery through signed manifests.
FR-023	SHOULD	Support OpenFX compatibility for conventional VFX plugins in a later phase.
FR-024	SHOULD	Represent semantic audio sources and link them to visual entities or events when confidence permits.
FR-025	MUST	Export standard playable media while preserving requested color, timing, audio, and provenance properties.

8. Non-Functional Requirements

ID	Priority	Requirement
NFR-001	MUST	The application shall remain responsive during background AI/render operations; worker failure shall not crash the main UI process.
NFR-002	MUST	Project saves shall be transactional and recoverable after process termination, power failure, or partial worker failure.
NFR-003	MUST	The system shall avoid unnecessary CPU-GPU copies and support GPU-resident processing where hardware permits.
NFR-004	SHOULD	Intermediate results shall be cacheable by content and operation hash.
NFR-005	MUST	The architecture shall permit model/backend replacement without changing authoritative project semantics.
NFR-006	MUST	Critical capabilities shall have automated regression tests and benchmark baselines.
NFR-007	MUST	Security-sensitive dependencies and packages shall be pinned, scanned, and attributable.
NFR-008	MUST	The system shall be usable offline for supported local capabilities.
NFR-009	SHOULD	The project format shall remain forward-migratable and

ID	Priority	Requirement
		versioned.
NFR-010	SHOULD	Professional operations shall expose deterministic parameters and diagnostics where technically feasible.

9. Privacy Requirements

ID	Priority	Requirement
PRIV-001	MUST	Local-first shall be the default execution policy.
PRIV-002	MUST	No media, proxy, thumbnail, embedding, transcript, prompt, or project metadata shall leave the device unless policy explicitly permits it.
PRIV-003	MUST	Private-local mode shall disable third-party model APIs, cloud thumbnails, remote embeddings, remote transcription, and media-bearing telemetry.
PRIV-004	MUST	Offline mode shall technically disable outbound application networking rather than relying only on policy text.
PRIV-005	MUST	Crash reporting shall exclude media, prompts, transcripts, local paths, credentials, and other sensitive project content by default.
PRIV-006	MUST	Deletion shall remove project references and garbage-collect unreferenced derived assets/caches according to retention policy.
PRIV-007	SHOULD	Enterprise deployments shall support configurable retention and customer-controlled encryption keys.
PRIV-008	MUST	Internal provenance may be detailed; exported provenance shall be filtered to avoid leaking private local paths, usernames, or secret project information.
PRIV-009	MUST	Cross-project embeddings, caches, and agent memory shall remain isolated unless explicitly shared.
PRIV-010	MUST	Users shall be able to see whether each operation runs locally, in

ID	Priority	Requirement
		UCE cloud, in a private team environment, or through a third-party provider.

10. Security Requirements

ID	Priority	Requirement
SEC-001	MUST	Agents shall operate through capability APIs and shall not receive unrestricted shell, filesystem, network, or project mutation access.
SEC-002	MUST	Untrusted/community plugins and models shall not execute inside the main trusted UI process.
SEC-003	MUST	Plugin permissions shall follow least privilege and be explicitly declared.
SEC-004	MUST	Official plugins/models/releases shall be signed and hash-verified.
SEC-005	MUST	Unsafe arbitrary-code model serialization shall be rejected or isolated; safetensors/ONNX-style formats are preferred where applicable.
SEC-006	MUST	Network access for plugins and agents shall be denied by default and separately permissioned.
SEC-007	MUST	Model and plugin packages shall declare publisher, version, license, hashes, capabilities, permissions, and runtime requirements.
SEC-008	MUST	Updates shall be signed, verified, atomic, and rollback-capable.
SEC-009	MUST	UCE shall maintain a software bill of materials and dependency vulnerability scanning for releases.
SEC-010	MUST	Project content shall be treated as untrusted data with respect to agent instruction hierarchy to resist prompt injection.
SEC-011	MUST	Secrets/credentials shall use OS-native secure storage or equivalent secure key management.
SEC-012	MUST	Privileged operations shall generate audit records sufficient

ID	Priority	Requirement
		for post-event reconstruction.

10.1 Threat Model - Minimum Coverage

- Malicious plugin or model package.
- Prompt injection through subtitles, metadata, scripts, webpages, manifests, or connected content.
- Path traversal, symlink attacks, malformed archives, zip bombs, corrupted project bundles.
- Malformed media exploiting parsers or causing resource exhaustion.
- Unauthorized network exfiltration.
- Cross-project data access or embedding leakage.
- Credential theft.
- GPU/CPU/memory denial-of-service.
- Tampered update/package/signature.
- Cloud worker or third-party provider mishandling sensitive content.

11. Testing and Verification

ID	Priority	Requirement
TEST-001	MUST	Unit tests shall cover graph operations, hashes, identity rules, permissions, timeline math, asset resolution, and serialization.
TEST-002	MUST	Integration tests shall cover complete flows such as import -> entity selection -> tracking -> removal -> QA -> export.
TEST-003	MUST	GPU tests shall detect VRAM leaks, OOM recovery failures, driver incompatibilities, and worker crashes.
TEST-004	MUST	Regression suites shall run against a stable media corpus and block releases on defined quality regressions.
TEST-005	MUST	Security CI shall include malicious package, sandbox escape attempt, path traversal, malformed input, prompt injection, and exfiltration tests.
TEST-006	MUST	Privacy tests shall confirm zero external traffic in private-local/offline mode.
TEST-007	MUST	Failure-recovery tests shall include process kill, power-loss simulation, disk-full, corrupt cache, model crash, and network interruption.
TEST-008	MUST	Fuzz testing shall target parsers, manifests, project files, plugin RPC

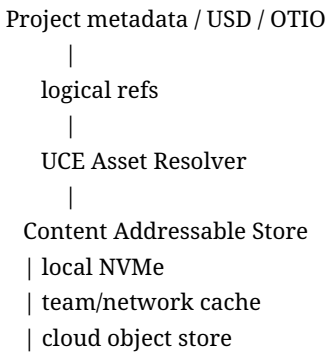
ID	Priority	Requirement
		surfaces, metadata, asset resolver inputs, and network messages.
TEST-009	SHOULD	Property-based tests shall verify invariants such as read-only plugin immutability and lock preservation.
TEST-010	MUST	Generative operations shall record enough parameters and model hashes for controlled reproducibility and regression analysis.

11.1 UCE-Bench

Item	Description
Edit Isolation Score (EIS)	Measures unintended change outside the requested edit region/state.
Dependency Completion Score (DCS)	Measures whether required dependent state such as shadows, reflections, audio, contact, and occlusion is correctly updated.
Entity Persistence Score (EPS)	Measures persistent identity across frames, occlusions, shots, representations, and edits.
Repair Efficiency Score (RES)	Measures defect improvement relative to the amount of content regenerated.
Instruction adherence	Whether the requested semantic transformation occurred.
Temporal consistency	Stability of appearance, geometry, motion, and state over time.
Geometry/physics correctness	Perspective, depth, support, collision, motion, and contact plausibility.
Performance	GPU-seconds, peak VRAM/RAM, storage, network transfer, latency, and estimated cost.

12. Data and Storage Architecture

UCE SHALL use content-addressable storage for immutable source and derived artifacts.



- Artifact identity SHOULD use a strong content hash such as SHA-256 or BLAKE3, subject to final ADR.

- Operation results SHOULD be keyed by operation type, input hashes, parameters, model version/hash, and deterministic settings.
- Compiled hardware-specific artifacts such as TensorRT engines SHALL be treated as disposable caches rather than canonical model state.
- Project snapshots SHOULD primarily update references and metadata rather than duplicate large binary media.
- Cache tiers MAY include VRAM, RAM, local NVMe, workstation cache, studio CAS, and cloud storage.

13. Universal Entity Identity (UEI)

UEI is a core UCE invention. Trackers, segmenters, recognizers, audio analyzers, and geometry systems provide evidence; they do not own identity.

```
UEI
|- persistent UUID
|- semantic class and labels
|- appearance/identity prototypes
|- observations (frames, masks, boxes, points)
|- representation references (pixels, depth, mesh, splats, skeleton, audio)
|- event references
|- relationships and dependencies
|- confidence history
|- provenance
```

ID	Priority	Requirement
UEI-001	MUST	Entity identity shall persist independently of a specific tracker/model.
UEI-002	MUST	An entity may have multiple simultaneous representations linked to one identity.
UEI-003	MUST	Identity confidence and evidence history shall be retained.
UEI-004	SHOULD	UCE shall support cross-shot re-identification and identity recovery after occlusion/disappearance.
UEI-005	MUST	Operations shall target stable Entity IDs rather than ephemeral masks or track indices.

14. World Graph and Dependency Model

- World Graph SHALL model entities, events, relations, temporal state, locks, dependencies, uncertainty, and provenance.
- Dependencies MAY cross modalities, for example vehicle motion -> engine audio or visual entity -> cast shadow.
- Changing authoritative state SHALL invalidate affected dependent computations.
- Dependencies SHOULD distinguish causal/required dependencies from weak inferred associations.
- Observed, estimated, and generated state SHOULD carry confidence/provenance labels.

Example:
vehicle_01
|- shadow_01
|- reflection_01
|- engine_audio_01
|- tire_contact_01
|- occlusion(person_03)
|- motion_blur

15. Media and GPU Pipeline

ID	Priority	Requirement
GPU-001	MUST	UCE shall support hardware-accelerated decode/encode where available.
GPU-002	SHOULD	GPU-resident surfaces shall pass between decode, AI, compositing, display, and encode with minimal host copies.
GPU-003	MUST	The GPU resource layer shall represent backend, format, ownership, synchronization, timestamps, and lifetime.
GPU-004	MUST	Backpressure limits shall prevent unbounded frame/tensor accumulation.
GPU-005	MUST	Model residency and eviction shall account for VRAM size, load cost, priority, and active work.
GPU-006	SHOULD	Vulkan/CUDA interoperability should be supported on NVIDIA where feasible; equivalent platform-native paths may be used elsewhere.

16. Model Runtime and Router

- Models SHALL be exposed as capabilities rather than hard-coded product dependencies.
- UCE SHOULD support PyTorch during research and portable/optimized runtimes such as ONNX/TensorRT where appropriate.
- Safetensors SHOULD be preferred for raw tensor weight storage where supported.
- Each model SHALL have a capability manifest containing identity, version, capabilities, runtime, weight format, hardware requirements, license, permissions, and benchmark metadata.
- The router SHOULD consider quality, latency, privacy, hardware, license, cost, resolution, and availability when selecting a backend.
- Local/private policy SHALL override quality-based routing when external execution is not permitted.

17. Plugin and Extension Architecture

Trust Class	Execution Rule
Built-in trusted component	Native, signed, highest privilege, internal review.
Signed native plugin	Isolated process where practical; explicit permissions.
Sandboxed community plugin	Restricted WASM/process sandbox; denied-by-default network/filesystem.
AI model plugin	Dedicated model worker/container with declared capabilities.
Agent plugin	May request only explicit registered capabilities; no direct project mutation.

- Plugin manifests SHALL declare GPU, network, filesystem, project-write, model-execution, and other permissions.
- Plugin crashes SHALL not terminate the main editor process.
- OpenFX compatibility SHOULD be investigated as a conventional VFX compatibility layer.

18. Agent Architecture

```
Director Agent
|- Editor Agent
|- Vision Agent
|- VFX Agent
|- Geometry/3D Agent
|- Physics Agent
|- Audio Agent
|- Continuity Agent
|- Rights/Provenance Agent
`- Quality Agent
```

- Agents plan and propose operations; they do not directly own authoritative state.
- Operation requests SHALL be validated by security/privacy policy before execution.
- Agent frameworks SHALL remain replaceable adapters; UCE's operation semantics must not depend on a specific third-party framework.
- Workflow durability SHALL be independent of agent reasoning.

19. Audio World

- Audio sources SHALL be representable as semantic entities with IDs, class, time range, confidence, and optional spatial estimates.
- Audio entities MAY link to visual entities or events.
- Supported future operations include separation, removal, replacement, spatialization, dialogue localization, Foley generation, and acoustics transformation.
- Cross-modal dependency updates SHOULD remove or replace semantically linked audio when the corresponding visual entity/event changes.

20. Quality and Self-Repair

```
Generate / Transform
        |
```

```
Quality Critics
      |
localize defect (entity + frame range + region + type)
      |
Repair Planner
      |
smallest valid repair
      |
re-check
```

- Quality critics SHOULD include temporal, identity, geometry, anatomy, physics, lighting, reflection, audio-sync, instruction, and preservation critics.
- Critic output SHALL include score, confidence, affected entity, frame range, region, problem type, and severity when available.
- Repair SHOULD preserve known-good content and regenerate only the smallest necessary region/time interval.
- Quality maps SHOULD be storable as first-class project data.

21. Provenance and Auditability

- UCE SHALL retain internal lineage: operation ID, input hashes, model ID/version/hash, parameters, seed, runtime, output hash, actor/agent, QA result, and timestamp.
- Exports MAY include privacy-filtered C2PA-compatible provenance.
- External provenance SHALL NOT expose local paths, usernames, internal server names, private prompts, or secret asset locations.
- Audit logs SHALL be tamper-evident where practical in enterprise deployments.

22. Interfaces and Standards

Standard/Technology	Intended Role
OpenUSD	Scene hierarchy, geometry, transforms, cameras, lights, materials, animation, custom UCE schemas.
OpenExec / equivalent	Dependency-aware evaluation and invalidation candidate.
OpenTimelineIO	Editorial structure and timeline interchange.
FFmpeg	Media decode/encode/container processing.
OpenColorIO	Color management.
OpenEXR	HDR/deep professional image interchange.
OpenImageIO	Image I/O and caching.
OpenVDB/NanoVDB	Sparse volumes.
MaterialX	Material/look representation.
OpenFX	Conventional VFX plugin compatibility candidate.
C2PA	External provenance/content credentials.
Safetensors	Preferred safe tensor storage where supported.
ONNX	Portable inference graph format where appropriate.

23. V0 Scope and Acceptance Criteria

23.1 V0 Objective

Prove the UCE architecture through one narrow but production-disciplined workflow: import -> understand/select -> persist identity -> track -> remove -> verify -> repair -> export.

ID	Priority	Requirement
V0-001	MUST	Import at least MP4/H.264 video with one audio stream and preserve source metadata needed for export.
V0-002	MUST	Create a transactional UCE project with CAS-backed source reference.
V0-003	MUST	Allow point/box based semantic selection on a frame.
V0-004	MUST	Create a stable UEI entity and propagate its mask/track through the selected shot.
V0-005	MUST	Remove the entity using reconstruction-first temporal processing.
V0-006	MUST	Preserve unrelated regions within defined benchmark tolerance.
V0-007	MUST	Run automated temporal/preservation checks and identify low-confidence defects.
V0-008	SHOULD	Repair detected localized defects automatically when a supported repair path exists.
V0-009	MUST	Export playable video with source audio preserved unless explicitly changed.
V0-010	MUST	Complete the entire workflow in private-local mode with no external network traffic.
V0-011	MUST	Recover the project after worker crash or application restart without losing committed state.
V0-012	MUST	Log model versions, operation parameters, input/output hashes, and QA results.

23.2 V0 Release Gate

- All MUST V0 requirements pass.
- Offline privacy test records zero unauthorized outbound traffic.
- Security tests for malformed package/project input and privilege boundaries pass.
- Regression suite shows no defined critical benchmark regression.
- No known data-loss defect in transactional save/recovery.

- No unhandled main-process crash caused by AI worker failure.
- Documented licenses for all distributed dependencies/models.

24. Roadmap

Phase	Primary Outcome
Phase 0	Research dossier, standards, licenses, threat model, benchmark design.
Phase 1	Master architecture, ADRs, schema contracts, repo structure.
Phase 2	Media core, CAS, transactional project state, GPU surfaces.
Phase 3	Semantic perception, UEI, tracking, V0 removal workflow.
Phase 4	Edit locks, preservation engine, self-repair.
Phase 5	Replacement/insertion and dependency-aware updates.
Phase 6	Geometry/world reconstruction and OpenUSD world editing.
Phase 7	3D/VFX/procedural systems and OpenFX compatibility.
Phase 8	Audio World and cross-modal dependency reasoning.
Phase 9	Event/action editing and explicit motion constraints.
Phase 10	Director/specialist agents, model router, cloud execution.
Phase 11	Plugin ecosystem, collaboration, enterprise/private deployment.
Phase 12	Editable generated worlds and broader universal creative workflows.

25. Risk Register

Risk	Severity	Why It Matters	Mitigation
Scope explosion	Critical	Trying to replace every creative application before proving core architecture.	Strict phased scope; V0 limited to one semantic removal workflow.
Model dependency/licensing	High	Research models may be non-commercial, disappear, or change license.	Capability abstraction, license registry, multiple backends.
Identity drift	Critical	Breaks continuity and semantic editing.	UEI as independent authority; multi-evidence identity memory.
GPU cost/latency	High	Threatens free-first product economics.	Local-first, reconstruction-first,

Risk	Severity	Why It Matters	Mitigation
			caching, region repair, model routing.
Privacy leakage	Critical	Unreleased/sensitive media could leave device.	Policy engine, offline enforcement, network-deny default, privacy tests.
Malicious plugins/models	Critical	Could steal media/credentials or execute arbitrary code.	Signing, sandboxing, process isolation, safe formats, permissions.
Generative unreliability	High	Makes professional use unpredictable.	Locks, deterministic paths, critics, localized repair, benchmarks.
Data loss/corruption	Critical	Creative projects are high-value.	CAS, immutable assets, transactional snapshots, recovery testing.
Architecture over-complexity	High	Could slow implementation and onboarding.	Minimal V0, clear contracts, ADRs, modular adapters.
Standards mismatch	Medium	Open standards may not encode all UCE semantics.	Custom UCE schemas/sidecar indexes; preserve interoperability.

26. Traceability Matrix

Goal	Requirements	Architecture Component	Primary Verification
Local-first privacy	FR-017, PRIV-001..010	Privacy Policy Engine / Network Layer	Offline network test + integration test
Persistent entity identity	FR-005, FR-006, UEI-001..005	UEI / Semantic World	Entity Persistence Score + tracking tests
Non-destructive editing	FR-010, FR-011	Operation Graph / Edit Locks	Undo/redo property tests + EIS
Secure extensibility	FR-022, SEC-001..009	Plugin/Model Runtime	Sandbox and supply-chain security tests
Self-repair	FR-013, FR-014	Quality Graph / Repair Planner	Repair Efficiency Score
Recoverable projects	NFR-002, V0-011	CAS + transactional snapshots	Crash/power-loss recovery tests
Dependency-aware edits	FR-012, FR-024	World Graph / Dependency Graph	Dependency Completion Score
Professional interoperability	FR-021, FR-023	Standards/adapters	Import/export/plugin compatibility tests

27. References

1. OpenUSD documentation and schema/composition architecture - Alliance for OpenUSD / Pixar.

2. OpenTimelineIO - Academy Software Foundation.

3. OpenColorIO - Academy Software Foundation.

4. OpenEXR - Academy Software Foundation.

5. OpenImageIO - Academy Software Foundation.

6. OpenVDB / NanoVDB - Academy Software Foundation.

7. MaterialX - Academy Software Foundation.

8. OpenFX - Academy Software Foundation / Open Effects Association.

9. FFmpeg project documentation.

10. C2PA Technical Specification and Content Authenticity Initiative SDKs.

11. Meta Segment Anything 2 (SAM 2) research/codebase.

12. Cutie / XMem-family video object segmentation research.

13. VGGT - visual geometry grounded transformer research/codebase.

14. Nerfstudio and gsplat for neural/gaussian scene representation.

15. 3D Gaussian Splatting and dynamic/4D Gaussian research.

16. TokenFlow and AnyV2V for temporally consistent video editing.

17. VBench / VBench-2.0 for video evaluation.

18. NVIDIA Video Codec SDK, CUDA/Vulkan interoperability, Triton Inference Server.

19. Temporal durable workflow concepts and OpenCue render-farm architecture.

20. Safetensors, ONNX, and TensorRT model/runtime documentation.

21. Blender, Vulkan, Qt/QML, and professional VFX pipeline documentation.

22. Foundational literature in computer vision, multiple-view geometry, physically based rendering, numerical optimization, signal processing, robotics/SLAM, computer graphics, animation, and distributed systems.

Appendix A - Proposed Repository Domains

```
uce/
|- core/
|- world/
|- identity/
|- operations/
|- media/
|- gpu/
|- vision/
|- models/
|- quality/
|- security/
|- privacy/
|- storage/
|- plugins/
|- agents/
|- audio/
|- ui/
`- tests/
```

Appendix B - Initial Architecture Decision Records

- ADR-001: Use OpenUSD as scene/world interoperability foundation.
- ADR-002: Use OpenTimelineIO for editorial timeline representation.
- ADR-003: Use content-addressable storage for immutable media/derived artifacts.
- ADR-004: Adopt local-first execution as default privacy policy.
- ADR-005: Universal Entity Identity is independent from any specific tracker/model.
- ADR-006: Agents may plan but may not directly mutate project state.
- ADR-007: Use reconstruction-first, generation-second for removal/recovery operations.
- ADR-008: Use signed capability manifests for models/plugins.
- ADR-009: Treat testing, privacy, and security as release gates.
- ADR-010: Preserve deterministic/manual controls alongside AI workflows.

Appendix C - Glossary

Item	Description
CAS	Content Addressable Store.
UEI	Universal Entity Identity.
World Graph	Semantic representation of entities, events, relationships, dependencies, uncertainty, and temporal state.
Operation Graph	Executable dependency graph of non-destructive transformations.
Edit Lock	Constraint that specifies state that must remain invariant during a transformation.
Quality Graph	Stored critic/defect state associated with project regions/entities/operations.
EIS	Edit Isolation Score.
DCS	Dependency Completion Score.
EPS	Entity Persistence Score.
RES	Repair Efficiency Score.
V0	First architecture-proving implementation milestone.
Local-first	Processing remains on-device unless explicitly authorized otherwise.